

# Graph Search Algorithms

Kanishk Goel

14 November 2025

## Contents

|          |                                      |          |
|----------|--------------------------------------|----------|
| <b>1</b> | <b>Free Primitives</b>               | <b>2</b> |
| <b>2</b> | <b>Generic Graph Search</b>          | <b>2</b> |
| <b>3</b> | <b>Breadth First Search</b>          | <b>2</b> |
| <b>4</b> | <b>Connected Components</b>          | <b>2</b> |
| <b>5</b> | <b>Depth First Search</b>            | <b>3</b> |
| <b>6</b> | <b>Topological Sort</b>              | <b>4</b> |
| <b>7</b> | <b>Strongly Connected Components</b> | <b>5</b> |
| 7.1      | Kosaraju . . . . .                   | 5        |

## 1 Free Primitives

Treat any algorithm with linear running time as something we can essential use for free.

## 2 Generic Graph Search

**Input:** An undirected/directed graph  $G = (V, E)$ , and a starting vertex  $s \in V$

**Output:** Identify the vertices of  $V$  reachable from  $s$  in  $G$ .

| Generic Graph Search                                                                                                                                                                      |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Input:</b> graph $G = (V, E)$ and vertex $s \in V$<br><b>Output:</b> a vertex is reachable from $s$ if and only if it is marked as “explored”                                          |
| mark $s$ as explored and all others unexplored<br><b>while</b> $\exists (v, w) \in E$ with $v$ explored and $w$ unexplored <b>do</b><br>choose some edge $(v, w)$<br>mark $w$ as explored |

## 3 Breadth First Search

Explores layer by layer, Layer 0 being  $s$  itself.

| Breadth First Search (BFS)                                                                                                                                                                                                                                                                                                                                                              |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| mark $s$ as explored, all other vertices as unexplored<br>$Q \leftarrow$ a queue data structure, init with $s$<br><b>while</b> $Q$ is not empty <b>do</b><br>remove the vertex from the front of $Q$ , call it $v$<br><b>foreach</b> edge $(v, w)$ in $v$ 's adjacency list <b>do</b><br><b>if</b> $w$ is unexplored <b>then</b><br>mark $w$ as explored<br>add $w$ to the end of queue |

Runtime -  $O(m + n)$

## 4 Connected Components

In an undirected graph  $G = (V, E)$ , a maximal subset  $S \subseteq V$  of vertices such that there is a path from any vertex in  $S$  to any other vertex in  $S$ .

UCC algorithm is used which takes undirected graph  $G = (V, E)$  in adjacency-list representation with  $V = \{1, 2, 3, \dots, n\}$

Postcondition:  $\forall u, v \in V, cc(u) = cc(v) \iff u, v$  are in the same connected component

| Undirected Connected Components (UCC)                                                                                                                                                                                                                                                                                                                                                                                                                                                                                 |  |  |  |  |
|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|--|--|
| <pre> mark all vertices as unexplored numCC ← 0 for i ← 1 to n do     if i is unexplored then         numCC ← numCC + 1                // new component         // call BFS         Q ← a queue data structure, init with s         while Q is not empty do             remove the vertex from the front of Q, call it v             foreach edge (v, w) in v's adjacency list do                 if w is unexplored then                     mark w as explored                     add w to the end of queue </pre> |  |  |  |  |

Runtime -  $O(m + n)$

## 5 Depth First Search

Takes one path and only backtracks when absolutely necessary.

| Iterative Implementation of DFS                                                                                                                                                                                                                                                                        |  |  |  |  |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|--|--|--|--|
| <pre> mark all vertices as unexplored S ← a stack data structure, init with s while S is not empty do     pop the vertex v from the front of S     if v is unexplored then         mark v as explored         foreach edge (v, w) in v's adjacency list do             push w to the front of S </pre> |  |  |  |  |

| Recursive Implementation of DFS                                                                                                                                                                                                                                                                        |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> <b>Func</b> DFS(<math>G, v</math>):     mark <math>s</math> as explored and all other as unexplored     <b>foreach</b> <math>edge(s, v)</math> in <math>s</math>'s adjacency list <b>do</b>         <b>if</b> <math>v</math> is unexplored <b>then</b>             DFS(<math>G, v</math>) </pre> |

Runtime -  $O(m + n)$

## 6 Topological Sort

### Topological Ordering

Let  $G = (V, E)$  be a directed graph. A topological ordering of  $G$  is an assignment  $f(v)$  of every vertex  $v \in V$  to a different number such that

$$\forall (v, w) \in E, f(v) < f(w)$$

- Topological orderings do not exist in directed cyclic graphs.
- Every DAG has a topological ordering.

### Topological Sort

DAG  $G = (V, E)$  in adjacency list representation is given a  $f$  value for every vertex constituting a topological ordering. It uses DFS-Topo which assigns every vertex reachable from a vertex  $s$  an  $f$ -value.

| TopoSort                                                                                                                                                                                                                                                                             |
|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> <b>Func</b> TopoSort(<math>G</math>):     mark all vertices as unexplored     <math>curLabel \leftarrow  V </math>     <b>foreach</b> <math>v \in V</math> <b>do</b>         <b>if</b> <math>v</math> is unexplored <b>then</b>             DFS-Topo(<math>G, v</math>) </pre> |

| DFS-Topo                                                                                                                                                                                                                                                                                                                                                                                  |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <pre> <b>Func</b> DFS-Topo(<math>G, s</math>):     mark <math>s</math> as explored     <b>foreach</b> <math>edge(s, v)</math> in <math>s</math>'s outgoing adjacency list <b>do</b>         <b>if</b> <math>v</math> is unexplored <b>then</b>             DFS-Topo(<math>G, v</math>)     <math>f(s) \leftarrow curLabel</math>     <math>curLabel \leftarrow curLabel - 1</math> </pre> |

$curLabel$  is a global variable, initialised to  $|V|$ . Runtime is  $O(m + n)$ .

## 7 Strongly Connected Components

A strongly connected component or SCC of a directed graph is a maximal subset  $S \subseteq V$  of vertices such that there is a directed path from any vertex in  $S$  to any other vertex in  $S$ .

Every directed graph can be viewed as directed acyclic graph built up from its SCCs.

### The SCC Meta-Graph is Directed Acyclic

Let  $G = (V, E)$  be a directed graph. Define the corresponding meta-graph  $H = (X, F)$  with one meta-vertex  $x \in X$  per SCC of  $G$  and a meta-edge  $(x, y)$  in  $F$  whenever there is an edge in  $G$  from a vertex in the SCC corresponding to  $x$  to one in the SCC corresponding to  $y$ . Then  $H$  is a directed acyclic graph.

Therefore, it is basically a compressed graph where each SCC of the graph is compressed into a node.

#### 7.1 Kosaraju

**Corollary:** For a directed graph  $G$ , let  $f(v), g(v)$  denote position of  $v \in V$  as computed by TopoSort algorithm for  $G$  and  $G^{rev}$  respectively. Also, let  $S_1, S_2$  denote two SCCs of  $G$  with an edge  $(v, w)$  with  $v \in S_1$  and  $w \in S_2$ .

$$\min_{x \in S_1} f(x) < \min_{y \in S_2} f(y).$$

$$\min_{x \in S_1} g(x) > \min_{y \in S_2} g(y).$$

We can see that the vertex  $x$  with  $f(x) = 1$  will be the source vertex i.e for two connecting SCCs the one with  $x$  will be the source of the directed edge. Therefore, for  $G^{rev}$ , it corresponds to a vertex  $y$  such that  $g(y) = 1$  and  $y$  resides in the ‘sink’ SCC. (Sink SCC is the SCC that connects to no other SCC). It is vital to identify these sink SCCs for Kosaraju.

| Kosaraju                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                          |
|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Input:</b> directed graph <math>G = (V, E)</math> in adjacency-list representation, with <math>V = \{1, 2, 3, \dots, n\}</math></p> <p><b>Output:</b> for every <math>v, w \in V</math>, <math>scc(v) = scc(w) \iff v, w</math> are in the same SCC of <math>G</math></p> <hr/> <p><math>G^{rev} \leftarrow G</math> with all edges reversed<br/> mark all vertices of <math>G^{rev}</math> as unexplored<br/> // computes <math>f(v)</math>'s magical ordering<br/> TopoSort(<math>G^{rev}</math>)<br/> // finds SCCs in reverse topological order<br/> mark all vertices of <math>G</math> as unexplored<br/> <math>numSCC \leftarrow 0</math><br/> <b>foreach</b> <math>v \in V</math> in increasing order of <math>f(v)</math> <b>do</b><br/>     <b>if</b> <math>v</math> is unexplored <b>then</b><br/>         <math>numSCC \leftarrow numSCC + 1</math><br/>         DFS-SCC(<math>G, v</math>)</p> |

| DFS-SCC                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                                              |
|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <p><b>Input:</b> directed graph <math>G = (V, E)</math> in adjacency list representation, and a vertex <math>s \in V</math>.</p> <p><b>Output:</b> every vertex reachable from <math>s</math> is marked as “explored” and has an assigned <math>scc</math>-value.</p> <hr/> <p>mark <math>s</math> as explored<br/> <math>scc(s) \leftarrow numSCC</math><br/> <b>foreach</b> edge <math>(s, v)</math> in <math>s</math>'s outgoing adjacency list <b>do</b><br/>     <b>if</b> <math>v</math> is unexplored <b>then</b><br/>         DFS-SCC(<math>G, v</math>)</p> |

Runtime:  $O(m + n)$  where  $m = |E| \wedge n = |V|$

Obvious Optimization - run DFS on the original graph, just go backwards on the edges.